N07: RESTful Web Services

Tags

> **“Love turns work into rest.”**
>
> - Teresa of Avila

Recap

For what seems like the longest time we have been trying to understand the motivations behind SOAP based web services. In this exploration we have encountered the necessary components for a sustainable web service framework. The two main takeaways from this are:

Representation:
The rules by which data and information is structured and presented between machines, processes and applications.

Standardisation:
The rules by which arbitrary machines can communicate, proknockercess and exchange representations.

Modularity:

The concept of functional separation, making sure aspects of machines and services are designed and behave without needless dependencies on other resources.

Optimising the Framework

By now we should have an appreciation for some of the redundancies of SOAP-based web services. In particular, the way messages are designed, data is represented and how services are located and accessed.

Roy Fielding's doctoral dissertation on Architectural Styles and the Design of Network-based Software Architectures described an alternative approach. In this work, six key constraints were proposed for web service design and implementation. Some of the key themes that have emerged during the study of `SOAP` may still be apparent here.

The RESTful Architecture

> 💡 **Note:** A *resource* refers to an entity or piece of information a client wants to interact with. This could be a user profile, an image, a dataset or some other serialisable object. Resources are identified by a *Uniform Resource Identifier* (*`URI`*).

A Simple Distinction.

Naively, using a web service requires two components:

- Service **operations**
  - The defined **processes** for querying, computing, or storing some information, based on a client request
- **Data** to use in service operations
  - Parameters sent in client requests, Server-side databases, etc

---

For now, we can think: **all other aspects** of a web service exist to make the exchange of data and the performance of operations between the client and server **practical**.

With this in mind, let's take another look at REST.

The SOAP Process

- A `wsdl` document **described** how we could **use** the operations offered by a web service.

    - **Service operations** (or method names in the case of `RPC`) were usually **defined** in the `wsdl` document.
    - The `wsdl` document also provides the service **endpoint** and the expected **input/output** of certain **operations**.
- Clients can use the service endpoint to **locate** these operations
- A SOAP message would then be sent to that endpoint.
    - The SOAP message carries data specifying the operation to use and any parameters needed to perform that operation.
    - The server responds with another SOAP message, carrying data specifying the expected return value (or possibly a fault)

Now the REST

Service operations are a combination of **hosted data** and **HTTP methods**:

Hosted data

- Hosted data is called a **resource.**
- **Resources** are identified by a `URI`

HTTP Methods

- Manipulate hosted data**.**
- Can be used by the client to:
    - **send** data to **create** new records or **update** existing data hosted by the server
    - **read** and **delete** existing data hosted on the server.

- Sent data, in a client request or server response, is called a **representation**

Uniform Interface

The uniform interface principle ensures that the interface is **consistent** and **predictable**, allowing clients to interact with resources in a **standardised** way. This

simplifies the client-server communication and makes it easier to develop and maintain the system.

For example, `SOAP`-based services predominantly use `HTTP` as the transport mechanism for messages. `HTTP` methods (`GET`, `POST`, `PUT`, `DELETE`) can be used to perform operations on resources. `GET` request to retrieve data, a POST request to create a new resource, a `PUT` request to update an existing resource, and a `DELETE` request to remove a resource.

There are a further **four** guiding principles to achieving a uniform interface:

---

1. Resource-based

The design principle where resources are identified and accessed through their unique URIs (Uniform Resource Identifiers). This means that each entity or piece of information that a client wants to interact with is identified as a resource with a distinct URI, distinguishing it from the concept of a representation.

The server responds with an HTTP 200 OK status code and a JSON representation in the body.**Example:**

Here is an example of a `GET` request for a RESTful service:

```
GET /api/users/123 HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer your_access_token
```

In this example, we are requesting the user with the ID `123` from the `/api/users` endpoint. The request is made to the `example.com` host and includes the necessary headers, such as the content type and authorisation token.

---

2. Manipulation of Resources through Representation

- A resource is simply the **URI** identifying some functional aspect of a web service.
- A **representation** is the data sent or received when interacting with a **resource**.

- Resources are **manipulated through** representations.

Clients send representations to the server to create or update a resource, and the server responds with representations to provide information about the resource or the outcome of the manipulation.

Consider the following example request-response:

Request



The *client* sends an `HTTP PUT` request to update a user with the ID `123` at the `/api/users` endpoint (URI). The request includes a JSON representation in the body, containing the updated user information.

```
PUT /api/users/123 HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer your_access_token

{
  "name": "John Doe",
  "email": "john.doe@example.com"
}
```

Response



The *server* responds with a `HTTP 200 OK` status code and a JSON representation in the body, confirming the successful update of the user *resource*. The response includes the updated user information, including the ID, name, and email representation.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
```

```
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com"
}
```

3. Self-descriptive messages

Messages exchanged between clients and servers contain all the necessary information for each recipient to understand the message without relying on any external context.

In practice, self-descriptive messages involve using standardised media types, such as JSON or XML, to represent data in requests and responses. These media types include metadata and semantics that describe the structure and meaning of the data. This metadata could include content type, MIME type or cache control headers (more on these later), for example.

Including this self-descriptive data removes any ambiguity regarding how each message should be processed or what kind of response the sender is trying to illicit.

By utilising self-descriptive messages and cache control headers, RESTful services promote loose coupling, interoperability, and efficient use of network resources.

Request:

```
GET /api/users/123 HTTP/1.1
Host: example.com
Accept: application/json
```



The client is sending a `GET` request to retrieve user information with the ID `123` from the `/api/users` endpoint.
The request includes the necessary headers, such as the host and the desired response format, which is specified as `application/json` in the Accept header.

Response:

```
HTTP/1.1 200 OK
Content-Type: application/json
Cache-Control: max-age=3600

{
  "id": 123,
  "name": "John Doe",
  "email": "john.doe@example.com"
}
```

💡 The server responds with `Content-Type` header, specifying the response body is in `application/json` format, and a `Cache-Control` headers, indicating the response can be cached by the client for a maximum of 3600 seconds (1 hour).

---

4. Hypermedia as the Engine of Application State (HATEOS)

This principle emphasises the use of **hypermedia** links to **navigate** and interact with resources.
In a HATEOAS-compliant system:

- Responses from the server contain hyperlinks
- Hyperlinks provide information about available actions or operations
- Those operations can be performed on the current resource.
- Hyperlinks are typically represented using standard formats such as HTML or JSON.

By including hyperlinks in the response, the server enables the client to navigate the application's state and discover available actions dynamically. This makes the client more independent and reduces coupling between the client and server.

Example:

Consider a RESTful API for a social media platform. A response for a user profile may include hyperlinks to **follow** the user, **view** their posts, or **send** them a message. The client can use these hyperlinks to interact with the user's profile without having prior knowledge of the API endpoints. HATEOAS promotes a more discoverable and self-descriptive API, allowing clients to navigate the application's resources and functionality effectively.

Request

```
GET /api/user/123 HTTP/1.1
Host: example.com
Accept: application/json
```

💡 In this example, the client sends a `GET` request to retrieve the user profile with the ID `123` from the `/api/user` endpoint. The request includes the necessary headers, such as the host and the desired response format, which is specified as `application/json` in the `Accept` header.

Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 123,
  "name": "John Doe",
  "email": "johndoe@example.com",
  "links": {
    "posts": "/api/user/123/posts",
    "follow": "/api/user/123/follow",
    "message": "/api/user/123/message"
  }
}
```



The response includes the user's ID, name, email, and hyperlinks (`links`) to perform various actions on the user's profile, such as accessing their posts, following them, or sending a message. These hyperlinks provide the client with information about the available actions or operations that can be performed on the user resource.

Client-Server Separation

A simple but essential principle. The client and server should be separate entities that communicate over a network.

This separation allows for scalability and flexibility by reducing the complexity of both client and server implementation. This way:

- The client is responsible for the **user interface** and **user experience**.
- The Server is responsible for **processing requests** and **managing resources**.
- Both client and server components can be developed independently.

Stateless

Similar to the self-descriptive message principle for uniform interfaces, the **stateless principle** implies requests from a client to a server should contain all necessary information for the server to understand and process the request. That is to say, there is no component to needed to track the ‘state’ of communications between client and server.

The idea is the combination of

- **HTTP Method**,
- **representation**
- **resource**

is enough information to know exactly what is being requested / responded.

The server does not store any client state between requests, which improves scalability and simplifies server implementation.

For example, using JSON Web Tokens (JWT) to include authentication information in each request, eliminating the need for the server to store session information.

REST Resource State:

Refers to the state of the resources themselves within a RESTful system. The state of a resource is its representation at a particular point in time. For example, a user profile resource may have different states, such as the user's name, email, and other attributes. The client can interact with the resource's state by performing operations on it, such as retrieving, creating, updating, or deleting the resource.

Cacheable

The cacheable principle allows responses from the server to be cached (stored) by the client. The server includes **cache control headers** in the response to specify the caching rules.

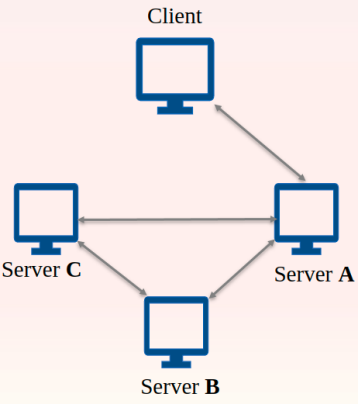
This reduces the need for future requests to the server, improves performance, and reduces network congestion.

For example, caching static resources like images, CSS files, and JavaScript files in the client's browser to avoid unnecessary server requests.

Layered System

The layered system principle allows the architecture to be composed of multiple layers, with each layer providing a specific set of functionality (Think of the OSI model or distributed systems in general).

Each layer can be managed independently and can have its own set of rules. This promotes **modularity**,



scalability, and **ease of maintenance**.

Within the REST architecture, a service should be able to receive a request on one server, retrieve stored data from a second server and manage client authentication via a third server.

An example is a typical web application architecture with a presentation layer, business logic layer, and data access layer, where each layer has its own responsibilities and can be developed and maintained separately.

Code on Demand (optional)

The code on demand principle is optional and not commonly used. It allows the server to send executable code to the client, allowing the client to extend its functionality dynamically. This principle is rarely used in practice but can be beneficial in certain scenarios.

An example is a JavaScript function being sent from the server to the client to be executed within the client's browser, enhancing the client's capabilities.

Request:

```
GET /api/code HTTP/1.1
Host: example.com
Accept: application/javascript
```



In this example, the client sends a `GET` request to retrieve executable code from the `/api/code` endpoint.
The request includes the necessary headers, such as the host and the desired response format, which is specified as `application/javascript` in the `Accept` header.

Response:

```
HTTP/1.1 200 OK
Content-Type: application/javascript

function calculateSum(a, b) {
  return a + b;
}
```



The server responds with a `Content-Type` header, specifying that the response body contains executable JavaScript code.
In this example, the response includes a JavaScript function `calculateSum` that can be executed within the client's browser, enhancing the client's capabilities.

Adhering to Constraints

Note Code on Demand is the only optional constraint of the 6 REST design principles. Any web service or API which violates any of the first five constraints cannot technically be referred to as RESTful.

Hopefully, the difference between SOAP and REST are a little clearer by this point. It should be easier to see how SOAP is a strict protocol while REST is an architectural style.

HTTP Methods

HTTP methods, also known as HTTP verbs, are used in the context of RESTful web services to perform different operations on resources. They correspond to the CRUD (Create, Read, Update, Delete) operations commonly used in database and application development.

Here are some examples of how HTTP methods correspond to CRUD operations:

GET: Used to retrieve (Read) a representation of a resource. For example:

- Request: `GET /api/users/123 HTTP/1.1`
- Response: `HTTP/1.1 200 OK`

POST: Used to create (Create) a new resource. For example:

- Request:

```
POST /api/users HTTP/1.1
Content-Type: application/json

{
  "name": "John Doe",
  "email": "john.doe@example.com"
}
```

- Response: `HTTP/1.1 201 Created`

PUT: Used to update (Update) an existing resource. For example:

- Request:

```
PUT /api/users/123 HTTP/1.1
Content-Type: application/json

{
  "name": "Jane Smith",
  "email": "jane.smith@example.com"
}
```

- Response: `HTTP/1.1 200 OK`

DELETE: Used to delete (Delete) a resource. For example:

- Request: `DELETE /api/users/123 HTTP/1.1`
- Response: `HTTP/1.1 204 No Content`

These HTTP methods, along with other HTTP headers and status codes, allow clients to interact with RESTful web services and perform various operations on the resources they represent.

Components of Representations and Resources

The Accept Header

The `Accept` header is an HTTP header included in a client request to specify the preferred media types (content types) for the response. It tells the server what kind of **representation** the client can process or expects in response.

For example, a client can include the `Accept` header with the value `application/json` to indicate that it prefers to receive the response in JSON format. If the server supports JSON and can provide the response in that format, it will include the appropriate `Content-Type` header in the response to indicate that the response body is in JSON.

The `Accept` header is useful in scenarios where the server offers multiple representations (e.g., JSON, XML) of the same resource, and the client can specify its preference.

Example:

```
GET /api/users/123 HTTP/1.1
Host: example.com
Accept: application/json
```

In this example, the client sends a `GET` request to retrieve user information with the ID `123` from the `/api/users` endpoint. The client includes the `Accept` header with the value `application/json` to indicate that it prefers to receive the response in JSON format

File Extension Format Specifier

In the context of REST resources, a file extension style format specifier refers to including a file extension in the `URI` to indicate the preferred representation format for a resource.

For example, if a client wants to retrieve a user resource in JSON format, it can include the file extension ".json" in the `URI`, such as `/api/users/123.json`. Similarly, if the client prefers `XML` format, it can include the file extension ".xml" in the `URI`, such as `/api/users/123.xml`.

By including a file extension in the `URI`, the client can explicitly specify its preferred representation format for the requested resource. The server can then use this information to provide the appropriate representation in the response.

It's worth noting that using file extensions in REST resources is not a strict requirement and is often optional. Alternatively, clients can specify their preferred representation format using the `Accept` header in the request.

Query String Parameters

In the context of REST, a query string parameter is a component of a URL that is used to pass additional information to the server. It is appended to the URL after a question mark "`?`" and consists of one or more key-value pairs separated by ampersands "`&`".

Query string parameters are commonly used to filter, sort, or paginate resources in RESTful APIs. The server can use these parameters to modify the response based on the client's requirements. For example, a client can include a query string parameter to retrieve only a subset of records from a database or to specify the sorting criteria for a list of resources.

Here is an example of a URL with query string parameters:

```
GET /api/users?role=admin&status=active&page=1&limit=10
```

In this example, the client is requesting a list of users from the `/api/users` endpoint. The query string parameters include "`role=admin`", "`status=active`", "`page=1`", and "`limit=10`". These parameters can be used by the server to filter the users based on their role and status, and to paginate the results by returning the first page with a limit of 10 users per page.

Query string parameters provide a flexible and standardized way for clients to customize the behavior of RESTful APIs and retrieve specific subsets of resources.

POST vs. PUT

When designing a RESTful API, you should use the `POST` method when creating a new resource on the server. This means that the server will generate a new URI for the resource and include it in the response.

On the other hand, you should use the `PUT` method when updating an existing resource on the server. This means that the client will send the updated representation of the resource to the server, and the server will replace the existing resource with the new representation.

In summary:

- Use `POST` to create a new resource.
- Use `PUT` to update an existing resource.

It's important to note that `PUT` is idempotent, meaning that multiple identical requests will have the same effect as a single request. On the other hand, `POST` is not idempotent, and multiple identical requests may result in multiple resource creations.

Remember to always consider the semantics and requirements of your API when deciding whether to use `POST` or `PUT` for creating or updating resources.
